

Підключення до SmartID

Призначення: сервіс надає клієнтам можливість використання хмарного КЕП

URL-шлях: <https://acsk.privatbank.ua/cloud/api/back>

- Опис
 - Зашифрований об'єкт (запит)
 - Приклад
 - Зашифрований об'єкт (відповідь)
 - Приклад
 - Підписаний об'єкт
 - Приклад
 - Помилки
 - Отримання сертифіката сервісу підписів
 - Відповідь успішна
 - Response
 - Отримання ідентифікатора сесії
 - Запит
 - Опис відповіді
 - Відповідь успішна
 - Запит на формування підпису
 - Запит
 - Опис відповіді
 - Відповідь успішна
 - Примітка
 - Передача даних у Приват24 моб. через QR-код
 - Формування QR-коду
 - Запит статусу формування підпису
 - Запит
 - Опис відповіді
 - Відповідь успішна
- Бібліотека
 - Приклад налаштування бібліотеки
 - Завантаження бібліотеки
 - Налаштування бібліотеки
 - Шифрування
 - Формування хешу
 - Об'єднання оригінальних даних з підписом
 - Перевірка підпису
- Бібліотека JS
 - Налаштування бібліотеки
 - Приклад налаштування бібліотеки для шифрування
 - Шифрування
 - Формування хешу
- Формування запитів

Опис

Зашифрований об'єкт (запит)

Формат даних: JSON

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад
	authData		Зашифрований сесійний ключ	String (Base64)	Так	
	encryptedData		Зашифровані дані	String (Base64)	Так	

Приклад

```
1 {
  "authData": "MIIC4gYJKoZIhvcNAQcDoIIC0zCCAsQDDDTQmtCd0VQS0xNDM2MDU3MC0yMzFLvxKv3b3kDALWNyJSg==",
  "encryptedData": "AAAAAAAAAADcJ48ukY5oMI8vtQv7yVBCKh2TLQNHIVmVwx+DGeFTQzyOLQQjqOU6iZ/9c2Zvr"
}
```

Зашифрований об'єкт (відповідь)

Формат даних: JSON

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад
	encryptedData		Зашифровані дані	String (Base64)	Так	

Приклад

```
1 {
  "encryptedData": "MIIC4gYJKoZIhvcNAQcDoIIC0zCCAsQDDDTQmtCd0VQS0xNDM2MDU3MC0yMzFLvxKv3b3kDALWNyJSg=="
}
```

Підписаний об'єкт

Формат даних: JSON

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад даних
	signedData		Підписані дані	String (Base64)	Так	

Приклад

```
1 {
  "signedData": "MIIC4gYJKoZIhvcNAQcDoIIC0zCCAsQDDDTQmtCd0VQS0xNDM2MDU3MC0yMzFLvxKv3b3kDALWNyJSg=="
}
```

Помилки

Формат даних: JSON

Актуальна версія: v1
Дефолтна версія: v1

Версія 1

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад даних
	errorCode		Код помилки	int	Так	
	errorMessage		Опис помилки	String	Так	
	additionalErrorCode		Додатковий код помилки	int	Ні	
	additionalErrorMessage		Додатковий опис помилки	String	Ні	

Отримання сертифіката сервісу підписів

Формат запиту - GET

URL: /get-certificates

Актуальна версія: v1
Дефолтна версія: v1

Формат відповіді

Формат даних: JSON

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад
	certificates		Список сертифікатів сервісу підписів	List<String> (Base64)	Так	

Відповідь успішна

Response

1	{"certificates":["MIIF9jCCBZ6gAwIBAgIUk2x9+a0...i+33Ft4lqa2lC4D9ISdKHspzDxqAhcQ5xpr3fc=="]}]}
---	---

Отримання ідентифікатора сесії

Формат запиту: POST

Формат даних: JSON

URL: /acquire-operation-id

Актуальна версія: v1

Дефолтна версія: v1

Опис запиту

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад даних
	clientId		Уникальный идентификатор PREFIX_UUID PREFIX - идентификатор системы присылающей запросы	String	Так	DIYA_550E8400-E29b-41D4-A716-446655440000

Запит

Request
{"clientId":"DIYA_550E8400-E29b-41D4-A716-446655440000"}

Далі запит шифрується та оформляється за схемою "Зашифрований об'єкт (запит)"

Опис відповіді

Родительский тэг	Тег	Атрибут	Описание	Формат данных	Обязательное	Пример данных
	operationId		Идентификатор сессии	String (Base64) 88 символов	Да	

Відповідь успішна

Response
{"operationId":"weyMfXq32W0ukwDcpHTjZa/Alyk4X+0lyDcYVtDool18Lwd6jZDi1Z0osF5/QEj5tuPrFeQQDJAAEIWDcYVtD=="}

У відповіді отримуємо дані щодо схеми "Зашифрований об'єкт (відповідь)"

Після розшифрування даних отримуємо "Підписаний об'єкт"

Після перевірки підпису отримуємо вже об'єкт, що містить ідентифікатор сесії.

Запит на формування підпису

Формат запиту POST

Формат даних: JSON

URL: /sign

Актуальна версія: v1

Дефолтна версія: v1

Опис запиту

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад даних
	clientId		Унікальний ідентифікатор PREFIX_UUID PREFIX - ідентифікатор системи, що надсилає запити	String	Так	DIYA_550E84 00-E29b- 41D4-A716- 446655440000
	operationId		Ідентифікатор сесії	String (Base64) 88 символів	Так	
	time		Час формування запиту	String (yyyy-MM- dd HH:mm:ss)	Так	2019-09-16 10:21:45
	originatorD escription		Ідентифікатор системи в якій було сформовано запит	String	Так	DIYA
	operationD escription		Ідентифікатор операції в системі яка сформувала запит	String	Hi	
	operationD escriptions		Список найменувань документів, які будуть підписані	List<String>	Hi	["Договор залога", "Акт выполненных работ"]
	hashes		Список хешів від документів які необхідно підписати	List<String> (Base64) * Максимальний розмір списку == 40	Так	
	signatureAl gorithmNa me		Назва алгоритму підпису	const String: DST U4145	Так	
	signatureF ormat		Формат запитованого підпису	const String: PKC S7	Так	
	callbackUrl* **		URL для повідомлення, сервіс підпису відправить на цю адресу дані для отримання підпису після того, як його буде сформовано. *** При формуванні callbackUrl потрібно використовувати тільки https зі стандартним портом: 443	String	Нет	
	cmsFormat		Формат підпису 0 - базовий формат за замовчуванням (CADES-T) 1 - розширений формат (CADES-X-LONG)	int	Hi	0

Запит

Request

```
{
  "clientId": "DIYA_550E8400-E29b-41D4-A716-446655440000",
  "operationId": "WeyMfXq32W0ukwDcpHTjZa/Alyk4X+0lyDcYVtDool18Lwd6jZDi1Z0osF5/QEj5tuPrFeQQDJAAEIWDcYVtD==",
  "time": "2019-09-16 10:21:45",
  "originatorDescription": "DIYA",
  "operationDescription": "QAZWSX123",
  "hashes": ["yk4X+0lyDcYVtDool18Lwd6jZDi1Z0osF5/QEj5tuPr==", "Dool18Lwd6jZDi1Z0yk4X+0lyDcYVtosF5/QEj5tuPr=="],
  "signatureAlgorithmName": "DSTU4145",
  "signatureFormat": "PKCS7"
}
```

Далі запит шифрується та оформляється за схемою "Зашифрований об'єкт (запит)"

Опис відповіді

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад даних
	status		Статус обробки запиту	int на даний момент можливі такі статуси: 1 - операція розпочата	Так	

Відповідь успішна

Response

```
{"status":1}
```

У відповіді отримуємо дані щодо схеми "Зашифрований об'єкт (відповідь)"

Після розшифрування даних отримуємо "Підписаний об'єкт"

Після перевірки підпису отримуємо вже об'єкт, що містить статус обробки запиту.

Примітка

Якщо в запиті було вказано параметр callbackUrl, то після формування підпису сервіс підписів відправить на вказаний URL запит у форматі:

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад даних
	clientId		Унікальний ідентифікатор PREFIX_UUID PREFIX - ідентифікатор системи, що надсилає запити	String	Так	DIYA_550E8400-E29b-41D4-A716-446655440000
	operationId		Ідентифікатор сесії	String (Base64) 88 символів	Так	
	errorCode		Код помилки	int	Ні	
	errorMessage		Опис помилки	String	Ні	

Request
<pre>{ "clientId": "DIYA_550E8400-E29b-41D4-A716-446655440000", "operationId": "WeyMfXq32WoukDcpHTjZa/Alyk4X+OlyDcYVtDool18Lwd6jZDi1Z0osF5/QEj5tuPrFeQQDJAAEIWDcYVtD==" }</pre>

Цей запит не шифрується

Передача даних у Приват24 моб. через QR-код

Опис запиту

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад даних
	clientId		Унікальний ідентифікатор PREFIX_UUID PREFIX - ідентифікатор системи, що надсилає запити	String	Так	DIYA_550E840 0-E29b-41D4- A716- 446655440000
	operationId		Ідентифікатор сесії	String (Base64) 88 символів	Так	
	stampClientId *		Унікальний ідентифікатор PREFIX_UUID PREFIX - ідентифікатор системи, що надсилає запити	String	Hi	DIYA_550E840 0-E29b-41D4- A716- 446655440000
	stampOperationId *		Ідентифікатор сесії	String (Base64) 88 символів	Hi	
	action		тип операції: auth - операція авторизації, під час передавання цього типу необхідно відобразити тексти для авторизації sign - операція підпису, під час передавання цього типу, необхідно відобразити тексти для підпису encrypt - операція шифрування ключа, під час передавання цього типу, необхідно відобразити тексти для шифрування документів decrypt - операція дешифрування ключа, під час передавання цього типу, необхідно відобразити тексти для дешифрування документів	String	Hi	
	userType		тип користувача: 1 - юр.особа 2 - фіз.особа	int	Hi	
	keyType		тип ключа: 0 - підпис 1 - печатка 2 - підпис + печатка	int	Hi	

* використовуються в тому випадку коли потрібно підтвердити за один раз формування підпису ключем підпису та ключем печатки

Формування QR-коду

Сервіс, який відправив запит на підпис, повинен сформувати URL:

Якщо було надіслано запит на формування підпису ключем підпису, то використовуємо просте посилання типу:

[https://www.privat24.ua/rd/kep?hash=rd/kep/{\"clientId\":\"clientId\",\"operationId\":\"operationId\",\"action\":\"action\",\"userType\":\"userType\",\"keyType\":\"keyType\"}](https://www.privat24.ua/rd/kep?hash=rd/kep/{\)

У тих випадках, коли необхідно формування декількох підписів, наприклад, ключем підпису і ключем печатки формуємо посилання типу:

[https://www.privat24.ua/rd/kep?hash=rd/kep/{\"clientId\":\"clientId\",\"operationId\":\"operationId\",\"stampClientId\":\"stampClientId\",\"stampOperationId\":\"stampOperationId\",\"action\":\"action\",\"userType\":\"userType\",\"keyType\":\"keyType\"}](https://www.privat24.ua/rd/kep?hash=rd/kep/{\)

Приклад посилання:
`https://www.privat24.ua/rd/kep?hash=rd/kep/{\"clientId\":\"DIYA_550E8400-E29b-41D4-A716-446655440000\",\"operationId\":\"WeyMfXq32WOukwDcpHTjZa/Alyk4X+OlyDcYVtDool18Lwd6jZDi1ZOosF5/QEj5tuPrFeQQDJAAEIWDcYVtD==\",\"action\":\"sign\",\"userType\":1,\"keyType\":1}`

или

`https://www.privat24.ua/rd/kep?hash=rd/kep/{\"clientId\":\"DIYA_550E8400-E29b-41D4-A716-446655440000\",\"operationId\":\"WeyMfXq32WOukwDcpHTjZa/Alyk4X+OlyDcYVtDool18Lwd6jZDi1ZOosF5/QEj5tuPrFeQQDJAAEIWDcYVtD==\",\"stampClientId\":\"DIYA_550E8400-E29b-41D4-A716-446655440000\",\"stampOperationId\":\"WeyMfXq32WOukwDcpHTjZa/Alyk4X+OlyDcYVtDool18Lwd6jZDi1ZOosF5/QEj5tuPrFeQQDJAAEIWDcYVtD==\",\"action\":\"sign\",\"userType\":1,\"keyType\":2}`

Сформований URL необхідно перетворити на QR-код і відобразити клієнту.

Запит статусу формування підпису

Формат запиту: POST

Формат даних: JSON

URL: /sign-status

Актуальна версія: v1

Дефолтна версія: v1

Опис запиту

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад даних
	clientId		Унікальний ідентифікатор PREFIX_UUID PREFIX - ідентифікатор системи, що надсилає запити	String	Так	DIYA_550E8400-E29b-41D4-A716-446655440000
	operationId		Ідентифікатор сесії	String (Base64) 88 символів	Так	

Далі запит шифрується та оформляється за схемою "Зашифрований об'єкт (запит)"

Якщо в запиті /sign не переданий параметр callbackUrl необхідно повторювати запит /sign-status з періодичністю в 1-2 сек. впродовж 20 сек. доки не буде отриманий підпис.

Запит

Request
<pre>{ "clientId": "DIYA_550E8400-E29b-41D4-A716-446655440000", "operationId": "WeyMfXq32WOukwDcpHTjZa/Alyk4X+OlyDcYVtDool18Lwd6jZDi1ZOosF5/QEj5tuPrFeQQDJAAEIWDcYVtD==" }</pre>

Опис відповіді

Батьківський тег	Тег	Атрибут	Опис	Формат даних	Обов'язкове	Приклад даних
------------------	-----	---------	------	--------------	-------------	---------------

status			Статус обробки запиту	int на даний момент можливі такі статуси: 0 - операція зареєстрована (під операцію було виділено ідентифікатор) 1 - операція розпочата (надіслано запит клієнту) 2 - операція оброблена (клієнт підтвердив запит) 3 - операція відхилена (клієнт відхилив запит) 4 - помилка (при обробці запиту клієнтом сталася помилка)	Так	
signatures			Список із результатами підписання	List	Hi	
	hash		Хеш документа	String (Base64)	Так	
	signature		Підпис	String (Base64)	Hi	
	errorCode		Код помилки	int	Hi	
	errorMessage		Текст помилки	String	Hi	

Відповідь успішна

Response
<pre>{ "status":2, "signatures":[{ "hash":"yk4X+0lyDcYVtDool18Lwd6jZDi1Z0osF5/QEj5tuPr==", "signature":"WeyMfXq32W0ukwDcpHTjZa/Alyk4X+0lyDcYVtDool18Lwd6jZDi1Z0osF5/QEj5tuPrFeQQDJAAEIWDcYVtD==" }, { "hash":"Dool18Lwd6jZDi1Z0yk4X+0lyDcYVtosF5/QEj5tuPr==", "errorCode":504, "errorMessage":" " }] }</pre>

У відповіді отримуємо дані щодо схеми **"Зашифрований об'єкт (відповідь)"**

Після розшифрування даних отримуємо **"Підписаний об'єкт"**

Після перевірки підпису отримуємо вже об'єкт, що містить статус обробки запиту.

Бібліотека

Приклад налаштування бібліотеки

Завантаження бібліотеки

Бібліотека EUSignJava.jar завантажується прикладними програмами засобами середовища виконання java (JRE).

Після завантаження бібліотека EUSignJava.jar автоматично розархівує необхідні для роботи криптографічні бібліотеки до каталогу, що вказується прикладними програмами. Це повинен бути або каталог, до якого було встановлено прикладну програму, або будь-який інший каталог, за наявності прав запису та виконання до нього.

Встановлення каталогу, в якому буде розархівовуватися:

```
import com.iit.certificateAuthority.endUser.libraries.signJava.*;

public class EUSignApplication extends Application{

    public EndUser endUser;

    @Override

    public void onCreate() {

        super.onCreate();

        EndUserResourceExtractor.SetPath(this.getFilesDir().getAbsolutePath());

        EndUserResourceExtractor.SetPreLoad(true);

        endUser = new EndUser();

    }

}
```

Примітка. Встановлення каталогу, в який буде розархівовуватися програма повинно відбуватися до створення об'єкту EndUser.

Примітка. Підтримка архітектури armeabi-v7a виконується за рахунок автоматичного використання бібліотек для архітектури armeabi (для зменшення розміру програми). При підключенні в проект нативних бібліотек сторонніх виробників, необхідно використовувати лише версію для архітектури armeabi. Використання сторонніх бібліотек з архітектурою armeabi-v7a призводить до неможливості автоматичного використання бібліотек архітектури armeabi та помилок при їх завантаженні.

Налаштування бібліотеки

```
private void init() throws Exception {
    endUser.SetUIMode(false);
    endUser.SetLanguage(EndUser.EU_RU_LANG);
    endUser.SetCharset("UTF-8");

    try {
        endUser.Initialize();
        EndUserModeSettings modeSettings = new EndUserModeSettings(false);
        modeSettings.SetOfflineMode(false);
        endUser.SetModeSettings(modeSettings);

        EndUserProxySettings proxySettings = endUser.CreateProxySettings();
        proxySettings.SetUseProxy(false);
        proxySettings.SetAnonymous(true);
        proxySettings.SetAddress("");
        proxySettings.SetPort("");
        proxySettings.SetUser("");
        proxySettings.SetPassword("");
        proxySettings.SetSavePassword(false);
        endUser.SetProxySettings(proxySettings);

        EndUserFileStoreSettings fileStoreSettings = endUser.CreateFileStoreSettings();
        fileStoreSettings.SetPath("/home/pblib/Storage");
        fileStoreSettings.SetSaveLoadedCerts(true);
        endUser.SetFileStoreSettings(fileStoreSettings);

        EndUserCMPSettings cmpSettings = endUser.CreateCMPSettings();
        cmpSettings.SetUseCMP(true);
        cmpSettings.SetAddress("http://acsk.privatbank.ua/services/cmp/");
        cmpSettings.SetPort("80");
        cmpSettings.SetCommonName("");
        endUser.SetCMPSettings(cmpSettings);
    }
}
```

```

EndUserTSPSettings tspSettings = endUser.CreateTSPSettings();
tspSettings.SetAddress("http://acsk.privatbank.ua/services/tsp/");
tspSettings.SetPort("80");
tspSettings.SetGetStamps(true);
endUser.SetTSPSettings(tspSettings);

EndUserOCSPSettings ocsSettings = endUser.CreateOCSPSettings();
ocsSettings.SetAddress("http://acsk.privatbank.ua/services/ocsp/");
ocsSettings.SetPort("80");
ocsSettings.SetUseOCSP(true);
endUser.SetOCSPSettings(ocsSettings);

EndUserLDAPSettings LDAPSettings = endUser.CreateLDAPSettings();
endUser.SetLDAPSettings(LDAPSettings);

} catch (Exception e) {
    LOG.error("[EUSign] Initialization ERROR:", e);
}
}

```

Шифрування

```

//Клієнт

//cert - сертифікат, який отримали з сервера підписів
byte[] serverCert = Base64.getDecoder().decode(cert);

//створюємо сесію клієнта та ключі які використовуються для шифрування даних
//900 - час життя сесії за секунди

EndUserSession clientSession = endUser.ClientDynamicKeySessionCreate(900, serverCert);

//шифруємо дані клієнта

//req - дані, які необхідно зашифрувати

String encryptedData = endUser.SessionEncrypt(clientSession, req);

//Готуємо сесію передачі на сервер

String authData = Base64.getEncoder().encodeToString(clientSession.GetData());

//Розшифрувати відповідь сервера

byte[] decrResp = endUser.SessionDecrypt(clientSession, serverEncryptedResponse);

```

Формування хешу

Хеш розраховується з документа який необхідно підписати. Для обчислення хешу використовується алгоритм GOST34311.

Метод обчислення хешу (class EndUser): public String Hash(byte[] data) throws Exception;

де data - дані, які необхідно підписати

метод повертає хеш документа як Base64 рядка.

Об'єднання оригінальних даних з підписом

// Для перевірки підпису необхідно спочатку об'єднати оригінальні дані з підписаним гешем (отриманим в результаті запиту /sign-status)

int signerIndex = 0; //Вхідний. Індекс підписувача

// sign - Підпис у вигляді BASE64-строки

// Отримання інформації про підписувача. Повертає інформацію про підписувача у вигляді BASE64-строки

String signer = endUser.GetSigner(signerIndex, sign);

```
// Отримання сертифікату підписувача
byte[] cert = endUser.GetSignerCertificate(signerIndex, sign);

// Створення порожнього підпису. Повертає підпис у вигляді BASE64-строки
// data - оригінальні дані
String emptySign = endUser.CreateEmptySign(data);

// Додавання інформації про підписувача до підпису. Повертає підпис у вигляді BASE64-строки
String signedData = endUser.AppendSigner(signer, cert, emptySign);
```

Перевірка підпису

```
// Для перевірки підпису необхідно додати до локального сховища бібліотеки кореневі сертифікати АЦСК (EndUserFileStoreSettings)
// Сертифікати доступні за посиланням https://acsk.privatbank.ua/certificates/acsk

//signed - підписаний об'єкт, тобто. дані, які потрібно перевірити (Base64)
EndUserSignInfo info = endUser.VerifyInternal(signed);

//Отримуємо з підписаного об'єкта оригінальні дані
String original = info.GetDataString();

//Отримуємо інформацію про підписувача
EndUserOwnerInfo owner = info.GetOwnerInfo();
String ownerOrgName = owner.GetSubjOrg();
String ownerOrgUnit = owner.GetSubjOrgUnit();

// ІПН того хто сформував підпис
String ipn = owner.GetSubjDRFOCode();

//ЄДРПОУ того, хто сформував підпис
String edrpou = owner.GetSubjEDRPOUCode();

//Отримуємо інформацію про центр сертифікації
String acskCn = owner.GetIssuerCN();
```

Бібліотека JS

Налаштування бібліотеки

Бібліотека автоматично завантажується браузером використовуючи мову сценаріїв JavaScript.

Перед використанням бібліотека повинна бути розміщена на сервері. На web-(HTML-)сторінку, де використовується бібліотека підключається наступним чином:

```
<script type="text/JavaScript" src="euscpt.js"></script>

<script type="text/JavaScript" src="euscpm.js"></script>

<script async type="text/javascript" src="euscpc.js"></script>
```

Після завантаження модуля бібліотеки буде автоматично викликана функція сповіщення про завантаження:

```
function EUSignCPModuleLoaded();
```

За замовчанням після завантаження бібліотека буде автоматично проініціалізована та буде викликана функція сповіщення про ініціалізацію:

```
function EUSignCPModuleInitialized([boolean] isInitialized);
```

Використання функцій бібліотеки можливе тільки після її завантаження та ініціалізації.

Доступ до функцій бібліотеки виконується за рахунок звернення до об'єкту типу EUSignCP:

```
var endUser = EUSignCP();
```

Примітка. Бібліотека виконується в основному потоці NodeJS та блокує його. Для зменшення навантаження на процесор під час запитів до серверів АЦСК рекомендується додатково встановити модуль sleep:

```
npm install sleep
```

Примітка. Через політику безпеки web-браузерів, взаємодія з серверами ЦСК можлива лише за умови реалізації механізму кросдоменної взаємодії. Для реалізації взаємодії між доменами, сервер, на якому розміщено бібліотеку повинен, реалізовувати механізм проху-сервісу. Бібліотека, робить синхронний запит (POST або GET) до сервера за протоколом HTTP у вигляді:

```
/proxy?address=http://ca.server.ua, де
```

/proxy – адреса проху-сервісу, встановлюється функцією бібліотеки SetXMLHTTPProxyService;

?address= - параметр, що передається в запиті, містить адресу на яку потрібно перенаправити запит бібліотеки;

<http://ca.server.ua> – адреса на яку направлено запит бібліотеки.

Приклад налаштування бібліотеки для шифрування

```
console.log("Creating new endUser");
const endUser = EUSignCP();

console.log("Setting proxy handler");
endUser.SetXMLHTTPProxyService("/proxyHandler");

endUser.SetErrorMessageLanguage(EU_UA_LANG);
endUser.SetCharset("UTF-8");
endUser.SetJavaStringCompliant(true);

console.log("Setting ModeSettings");
const modeSettings = endUser.CreateModeSettings();
modeSettings.SetOfflineMode(false);
endUser.SetModeSettings(modeSettings);

console.log("Setting ProxySettings");
const proxySettings = endUser.CreateProxySettings();
proxySettings.SetUseProxy(false);
proxySettings.SetAnonymous(true);
proxySettings.SetAddress("");
proxySettings.SetPort("");
proxySettings.SetUser("");
proxySettings.SetPassword("");
proxySettings.SetSavePassword(false);
endUser.SetProxySettings(proxySettings);

console.log("Setting FileStoreSettings");
const fileStoreSettings = endUser.CreateFileStoreSettings();
fileStoreSettings.SetPath("/cert");
fileStoreSettings.SetSaveLoadedCerts(true);
endUser.SetFileStoreSettings(fileStoreSettings);

console.log("Setting CMPSettings");
const cmpSettings = endUser.CreateCMPSettings();
cmpSettings.SetUseCMP(true);
cmpSettings.SetAddress("http://acsk.privatbank.ua/services/cmp/");
cmpSettings.SetPort("80");
cmpSettings.SetCommonName("");
endUser.SetCMPSettings(cmpSettings);

console.log("Setting TSPSettings");
const tspSettings = endUser.CreateTSPSettings();
tspSettings.SetAddress("http://acsk.privatbank.ua/services/tsp/");
tspSettings.SetPort("80");
tspSettings.SetGetStamps(true);
endUser.SetTSPSettings(tspSettings);
```

```
console.log("Setting OCSPSettings");
const ocsSettings = endUser.CreateOCSPSettings();
ocsSettings.SetAddress("http://acsk.privatbank.ua/services/ocsp/");
ocsSettings.SetPort("80");
ocsSettings.SetUseOCSP(true);
endUser.SetOCSPSettings(ocsSettings);

console.log("Setting LDAPSettings");
const IDAPSettings = endUser.CreateLDAPSettings();
endUser.SetLDAPSettings(IDAPSettings);
```

Шифрування

```
console.log("Getting server certificate");
const serverCertBase64 = "MIIF4DCCBYigAwIBAgIUk2x9+aOJHaEEAAAA...";
const serverCertBytes = clientEndUser.Base64Decode(serverCertBase64);

console.log("Creating client session");
const sessionExpireTime = 900;
const endUserClientSession = endUser.ClientDynamicKeySessionCreate(sessionExpireTime, serverCertBytes);

console.log("Encrypting data");
const dataToEncrypt = "Hello World";
const dataToEncryptBytes = endUser.StringToArray(dataToEncrypt);
const encryptedDataBytes = endUser.SessionEncrypt(endUserClientSession, dataToEncryptBytes);
const encryptedDataBase64 = endUser.Base64Encode(encryptedDataBytes);
console.log("Encrypted data", encryptedDataBase64);

console.log("Getting authData");
const authDataBytes = endUserClientSession.GetData();
const authDataBase64 = endUser.Base64Encode(authDataBytes);
console.log("AuthData", authDataBase64);

console.log("Sending authDataBase64 and encryptedDataBase64 to server");
const encryptedServerResponseBase64 = sendEncryptedRequest(authDataBase64, encryptedDataBase64)
console.log("Encrypted server response", encryptedServerResponseBase64);

console.log("Decrypting server response on client side");
const decryptResponseBytes = endUser.SessionDecrypt(endUserClientSession, encryptedServerResponseBytes);
const decryptedResponse = endUser.ArrayToString(decryptResponseBytes);
console.log("Decrypted response", decryptedResponse);
```

Формування хешу

```
console.log("Creating hash Base64");
const dataToHash = "Hello World";
const dataToHashBytes = endUser.StringToArray(dataToHash);
const hash = endUser.HashData(dataToHashBytes, true);
console.log("Hash Base64", hash)
```

Формування запитів

URL		Шифрування	Підпис тех. ключем
/get-certificates	Запит	-	-
	Відповідь	-	-
/acquire-operation-id	Запит	+	-
	Відповідь	+	+
/sign	Запит	+	-
	Відповідь	+	+
/sign-status	Запит	+	-

	Відповідь	+	+
--	-----------	---	---

* У разі виникнення помилки відповідь не шифрується та не підписується